SIEMENS

Blocks 1

Tags and Tag tables 2

TIA Portal Openness: API for automation of engineering workflows

Manual

21.00.00.00

12/2025

NOTICE

This document was downloaded from the following site:

https://docs.industrial-operations-x.siemens.cloud

Siemens assumes no liability for the accuracy, completeness, or timeliness of the data provided.

Use is at your own risk.

Legal information

Warning notice system

This manual contains notices you have to observe in order to ensure your personal safety, as well as to prevent damage to property. The notices referring to your personal safety are highlighted in the manual by a safety alert symbol, notices referring only to property damage have no safety alert symbol. These notices shown below are graded according to the degree of danger.

DANGER

indicates that death or severe personal injury **will** result if proper precautions are not taken.

WARNING

indicates that death or severe personal injury **may** result if proper precautions are not taken.

CAUTION

indicates that minor personal injury can result if proper precautions are not taken.

NOTICE

indicates that property damage can result if proper precautions are not taken.

If more than one degree of danger is present, the warning notice representing the highest degree of danger will be used. A notice warning of injury to persons with a safety alert symbol may also include a warning relating to property damage.

Qualified Personnel

The product/system described in this documentation may be operated only by **personnel qualified** for the specific task in accordance with the relevant documentation, in particular its warning notices and safety instructions. Qualified personnel are those who, based on their training and experience, are capable of identifying risks and avoiding potential hazards when working with these products/systems.

Proper use of Siemens products

Note the following:

⚠ WARNING

Siemens products may only be used for the applications described in the catalog and in the relevant technical documentation. If products and components from other manufacturers are used, these must be recommended or approved by Siemens. Proper transport, storage, installation, assembly, commissioning, operation and maintenance are required to ensure that the products operate safely and without any problems. The permissible ambient conditions must be complied with. The information in the relevant documentation must be observed.

Trademarks

All names identified by ® are registered trademarks of Siemens Aktiengesellschaft. The remaining trademarks in this publication may be trademarks whose use by third parties for their own purposes could violate the rights of the owner.

Disclaimer of Liability

We have reviewed the contents of this publication to ensure consistency with the hardware and software described. Since variance cannot be precluded entirely, we cannot guarantee full consistency. However, the information in this publication is reviewed regularly and any necessary corrections are included in subsequent editions.

Siemens Aktiengesellschaft
Digital Industries
Postfach 48 48
90026 NÜRNBERG
GERMANY

12/2025 Subject to change

Copyright © Siemens 2026.
All rights reserved

Table of contents

1 Blocks 7

1.1 Querying the 'Program blocks' group 7
1.2 Querying the system group for system blocks 7
1.3 Enumerating system subgroups 8
1.4 Enumerating user-defined block groups 10
1.5 Enumerating all blocks 11
1.6 Querying information of a block/user data type 12
1.7 Setting and removing protections from a block 14
1.8 Setting up write protection of blocks 17
1.9 Deleting block 18
1.10 Accessing Data blocks 19
1.11 Creating and Renaming group for blocks 20
1.12 Deleting group for blocks 21
1.13 Accessing attributes of all blocks 22
1.14 Creating a ProDiag-FB 27
1.15 Accessing supervisions and properties of ProDiag-FB 28
1.16 Assigning ProDiag FB for DBs & Tag Table 29
1.17 Reading ProDiag-FB blocks and attributes 32
1.18 Exporting ProDiag alarm message 33
1.19 Adding an external file 34
1.20 Generate source from block 35
1.21 Generating blocks from source 37
1.22 Generating from source of known source format 38
1.23 Deleting user data type 41
1.24 Accessing PLC system data types 42
1.25 Deleting an external file 42
1.26 Starting the block editor 43
1.27 Changing blocks using fingerprints 44
1.28 Generating/deleting blocks for user defined pages 46
1.29 Writing access for OB block priority attribute 49
1.30 Generating block/UDT from external source file in specific user group 50
1.31 Generating loadable file for blocks outside of software units 51

2 Tags and Tag tables 56

2.1 Starting the 'PLC Tags' editor 56
2.2 Querying system groups for PLC tags 56
2.3 Creating PLC tag table 57
2.4 Enumerating user-defined groups for PLC tags 58
2.5 Creating user-defined groups for PLC tags 59
2.6 Deleting user-defined groups for PLC tags 60

2.7 Enumerating PLC tag tables in a folder 61
2.8 Querying information from a PLC tag table 62
2.9 Reading the time of the last changes of a PLC tag table 64
2.10 Deleting a PLC tag table from a group 65
2.11 Enumerating PLC tags 66
2.12 Accessing PLC tags 66
2.13 Accessing PLC constants 69

Blocks

1

1.1 Querying the 'Program blocks' group

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
  See Connecting to the TIA Portal
- A project is open.
  See Opening a project
- A PLC is determined in the project.

Program code

Modify the following program code to query the group "Program blocks":

```
Copies the following program code to clipboard.

private static void GetBlockGroupOfPLC(PlcSoftware
plcsoftware)
//Retrieves the system group of a block
{
    PlcBlockSystemGroup blockGroup = plcsoftware.BlockGroup;
}
```

1.2 Querying the system group for system blocks

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
  See Connecting to the TIA Portal
- A project is open.
  See Opening a project

Program code:

Modify the following program code to determine the group created for system blocks by the system:

```
Copies the following program code to clipboard.
```

```
PlcSoftware plcSoftware = ...
foreach (PlcSystemBlockGroup systemGroup in
plcSoftware.BlockGroup.SystemBlockGroups)
{
    foreach (PlcSystemBlockGroup group in systemGroup.Groups)
    {
        PlcBlockComposition pbComposition = group.Blocks;
        foreach (PlcBlock block in pbComposition)
        {
            //add your code here
        }
    }
}
```

1.3 Enumerating system subgroups

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
  See Connecting to the TIA Portal
- A project is open.
  See Opening a project

Program code: Enumerating all system subgroups

Modify the following program code to enumerate the system subgroups of all system blocks:

```
 Copies the following program code to clipboard.
//Retrieves the system generated group for system blocks
private static void GetSystemgroupForSystemblocks(PlcSoftware
plcSoftware)
{
    PlcSystemBlockGroupComposition systemBlockGroups =
plcSoftware.BlockGroup.SystemBlockGroups;
    if (systemBlockGroups.Count != 0)
    {
```

```
        PlcSystemBlockGroup sbSystemGroup =
systemBlockGroups[0];
        foreach (PlcSystemBlockGroup group in
sbSystemGroup.Groups)
        {
            EnumerateSystemBlockGroups(group);
        }
    }
}
private static void
EnumerateSystemBlockGroups(PlcSystemBlockGroup
systemBlockGroup)
{
    foreach (PlcSystemBlockGroup group in
systemBlockGroup.Groups)
    {
        // recursion EnumerateSystemBlockGroups(group);
    }
}
```

Program code: Accessing a specific subgroup

Modify the following program code to access a specific subgroup:

```
Copies the following program code to clipboard.
private static void AccessSbGroup(PlcSystemBlockGroup
systemBlockGroup)
{
    PlcSystemBlockGroup group1 =
systemBlockGroup.Groups.Find("User group XYZ");
    PlcSystemBlockGroup group2 = group1.Groups.Find("User
group ZYX");
}
```

See also

Adding an external file

1.4 Enumerating user-defined block groups

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
  See Connecting to the TIA Portal
- A project is open.
  See Opening a project
- A PLC is determined in the project.

Application

Subgroups are taken into account recursively for enumeration.

Program code: Enumerating all groups

Modify the following program code to enumerate the user-defined block groups:



```
Copies the following program code to clipboard.
//Enumerates all block user groups including sub groups
private static void
EnumerateAllBlockGroupsAndSubgroups(PlcSoftware plcsoftware)
{
    foreach (PlcBlockUserGroup blockUserGroup in
plcsoftware.BlockGroup.Groups)
    {
        EnumerateBlockUserGroups(blockUserGroup);
    }
}
private static void
EnumerateBlockUserGroups(PlcBlockUserGroup blockUserGroup)
{
    foreach (PlcBlockUserGroup subBlockUserGroup in
blockUserGroup.Groups)
    {
        EnumerateBlockUserGroups(subBlockUserGroup);
        // recursion
```

```
    }
}
```

Program code: Accessing a group

Modify the following program code to access a selected user-defined block group:

```
Copies the following program code to clipboard.
//Gives individual access to a specific block user group
private static void AccessBlockusergroup(PlcSoftware
plcsoftware)
{
    PlcBlockUserGroupComposition userGroupComposition =
plcsoftware.BlockGroup.Groups;
    PlcBlockUserGroup plcBlockUserGroup =
userGroupComposition.Find("MyUserfolder");
}
```

1.5 Enumerating all blocks

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
  See Connecting to the TIA Portal
- A project is open.
  See Opening a project
- A PLC is determined in the project.

Application

Targeted access to a program block is possible if its name is known.

Program code: Enumerating all blocks

Modify the following program code to enumerate the blocks of all block groups:

```
Copies the following program code to clipboard.
private static void EnumerateAllBlocks(PlcSoftware
plcsoftware)
//Enumerates all blocks
```

```
{
    foreach (PlcBlock block in plcsoftware.BlockGroup.Blocks)
    {
        // Do something...
    }
}
```

Program code: Accessing a specific block

Modify the following program code to access a specific block:

```
Copies the following program code to clipboard.
private static void AccessASingleBlock(PlcSoftware
plcsoftware)
//Gives individual access to a block
{
    // The parameter specifies the name of the block
    PlcBlock block =
plcsoftware.BlockGroup.Blocks.Find("MyBlock");
}
```

1.6 Querying information of a block/user data type

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
  See Connecting to the TIA Portal
- A project is open.
  See Opening a project

Application

The TIA Portal Openness API supports the querying of the following information for program and data blocks and for user data types:

- Time stamp in UTC time format.

  You check the following with the time stamp:

  - When the block was last compiled.
  - When the block was last changed.

- "Consistency" attribute

  The "Consistency" attribute is set to "True" in the following cases:

  - The block has been successfully compiled.
  - The block has not been changed since compilation.
  - No changes that would require re-compilation have been made to external objects.
- Programming language used (program and data blocks only)
- Block number
- Block name
- Block author
- Block family
- Block title
- Block version

See also Blocks and types of the TIA Portal Openness object model for further information.

Program code

Modify the following program code to query the information listed above:



```
Copies the following program code to clipboard.
private static void GetPlcBlockInformation(PlcSoftware
plcSoftware)
{
    PlcBlock plcBlock =
plcSoftware.BlockGroup.Blocks.Find("MyBlock");
    // Read information
    DateTime compileDate = plcBlock.CompileDate;
    DateTime modifiedDate = plcBlock.ModifiedDate;
    bool isConsistent = plcBlock.IsConsistent;
    int blockNumber = plcBlock.Number;
    string blockName = plcBlock.Name;
    ProgrammingLanguage programmingLanguage =
plcBlock.ProgrammingLanguage;
    string blockAuthor = plcBlock.HeaderAuthor;
    string blockFamily = plcBlock.HeaderFamily;
    string blockTitle = plcBlock.HeaderName;
```

```
    System.Version blockVersion = plcBlock.HeaderVersion;
}
```

See also

Importing configuration data

1.7 Setting and removing protections from a block

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
  See Connecting to the TIA Portal
- A project is open.
  See Opening a project
- PLC is not online.

Application

You can use the TIA Portal Openness to set or remove the password protection of a block via the PlcBlockProtectionProvider class and the PlcBlockProtectionProvider service. The service PlcBlockProtectionProvider is accessible on blocks which fulfill the following conditions:

- Block is know-how protectable
- Block is a code block or a global DB
- Block must be compiled
- Block is supported or editable in the current PLC
- Block is not in readonly context
- Block is not know-how protected
- Block is not online
- Block is not a CPU-DB
- Block is not of classic encryption language, ProDiag or ProDiag-OB
- Block is not an encrypted imported classic block

In case the block doesn't fulfill all conditions a null reference is returned by GetService() method.

Protect a block

Use the Protect() method to set the password to protect the porgramming block with.

```
 Copies the following program code to clipboard.

void Protect(SecureString password)
```

Errors will occur for following cases:

- An attempt to protect an already protected block: An EngineeringTargetInvocationException will be thrown with the message "You can't protect an already protected object".
- An attempt to protect with an empty string as password: An EngineeringTargetInvocationException will be thrown with the message "Password was not specified".
- An attempt to protect a failsafe block when the failsafe-program is password protected: An EngineeringTargetInvocationException will be thrown.
- An attempt to protect a failsafe block when the block is not called: An EngineeringTargetInvocationException will be thrown.

Unprotect a block

Use the Unprotect() method to remove the password the porgramming block is protected with.

```
Copies the following program code to clipboard.

void Unprotect(SecureString password)
```

Errors will occur in the following cases:

- An attempt to unprotect an already unprotected block: An EngineeringTargetInvocationException will be thrown with the message "You can't unprotect an object without protection".
- An attempt to unprotect with wrong password: An EngineeringTargetInvocationException will be thrown with the message "The used password was refused".
- An attempt to protect with an empty string as password: An EngineeringTargetInvocationException will be thrown with the message "Password was not specified".

Program code

```
Copies the following program code to clipboard.

PlcBlock block = ...;
PlcBlockProtectionProvider protectionProvider =
block.GetService<PlcBlockProtectionProvider>();
if (protectionProvider != null)
{
// ...
// perform know-how protection related operations here
}
```

Check for invalid characters

You can use any characters including backspace, tab etc. to protect a block with the Protect() method. The password submitted to protect a block the password is a SecureString. You can use the GetInvalidPasswordCharacters() to check if the provided password has invalid characters.



```
Copies the following program code to clipboard.

SecureString CreatePasswordString(ProtectionProvider
protectionProvider, IEnumerable<char> contentCharacters)
{
    IList<char> invalidCharacters =
protectionProvider.GetInvalidPasswordCharacters();
    SecureString password = new SecureString();
    foreach(char ch in contentCharacters)
    {
        if (!invalidCharacters.Contains(ch))
        {
            password.AppendChar(ch);
        }
        else
        {
            // at least one of the content characters is not
valid
            // signal an error - e.g. throw an exception
            ...
        }
    }
    return password;
}
```

Errors will occur in the following cases:

- An attempt to unprotect an already unprotected block: An EngineeringTargetInvocationException will be thrown with the message "You can't unprotect an object without protection".
- An attempt to unprotect with wrong password: An EngineeringTargetInvocationException will be thrown with the message "The used password was refused".

- An attempt to protect with an empty string as password: An EngineeringTargetInvocationException will be thrown with the message "Password was not specified".

1.8 Setting up write protection of blocks

Description

Using Openness you can now setup write protection of blocks. This feature is provided via the PlcBlockWriteProtectionProvider service.

The workflow of configuring the protection of a block is as follows:

- Find the block supporting write protection.
- Get the PlcBlockWriteProtectionProvider service from the block
- Create a secure password string and make it read only
- Call the 'Define' action on the protection service with the password to set the password
- Call the 'Protect' action on the protection service with the password to protect the block given the correct password
- Call the 'Unprotect' action on the protection service with the correct password to temporarily unprotect the block
- Call the 'Change' action on the protection service with the correct password and 'null' as second parameter to remove any password protection from the block

Program Code

```
Code example for modifying the write protection
Copies the following program code to clipboard.

PlcBlock block1 =
PlcSoftware.BlockGroup.Blocks.Find("Block_1");
//get the protection service
var writeProtectionService =
block1.GetService<PlcBlockWriteProtectionProvider>();
//create a secure password
SecureString pwd = new SecureString();
pwd.AppendChar('1'); //repeat for all characters of the
password
pwd.MakeReadOnly()
//define the password on the service
writeProtectionService.Define(pwd);
//protect the block
writeProtectionService.Protect(pwd);
//unprotect the block
```

```
writeProtectionService.Unprotect(pwd);
//remove the password and the write protection from the block
writeProtectionService.Change(pwd, null);
```

1.9 Deleting block

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
  See Connecting to the TIA Portal
- A project is open.
  See Opening a project
- PLC is not online.

Program code

Modify the following program code to delete a block:

```
Copies the following program code to clipboard.
//Runs through block group and deletes blocks
private static void DeleteBlocks(PlcSoftware plcsoftware)
{
    PlcBlockSystemGroup group = plcsoftware.BlockGroup;
    //  or BlockUserGroup group = ...;
    for (int i = group.Blocks.Count - 1; i >= 0; i--)
    {
        PlcBlock block = group.Blocks[i];
        if (block != null)
        {
            block.Delete();
        }
    }
}
```

See also

Importing configuration data

1.10 Accessing Data blocks

Creating Data block snapshots

You can create snapshots via the Openness API in the following workflow:

- Using the navigator 'Interface' on a DataBlock and request the service 'ValueService'.
- Call 'CreateSnapshot' action on the value service to create a snapshot of the current online values and store it on the block.

Program Code

```
Code example for retrieving the value service and creating a
snapshot Copies the following program code to clipboard.

DataBlock block1 =
(DataBlock)PlcSoftware.BlockGroup.Blocks.Find("DataBlock_1");
//Getting the value service for the data block
ValueService valueService =
block1.Interface.GetService<ValueService>();
//Creating the snapshot for the data block
//the PLC must be in 'Online' mode
valueService.CreateSnapshot();
//Exporting a snapshot via the already existing functionality
InterfaceSnapshot interfaceSnapshot =
valueService.GetService<InterfaceSnapshot>();
FileInfo path = …;
interfaceSnapshot.Export(path, ExportOptions.None);
```

Loading snapshots as actual values

You can load an existing data block snapshot as actual value via the Openness API. Once a snapshot is stored for a DataBlock the values can be loaded back as actual values to the PLC with the following workflow:

- Using navigator 'Interface' on a DataBlock request the service 'ValueService'.
- If not yet done before, create a snapshot for the DataBlock.
- Call the 'LoadSnapshotAsActualValues' action on the value service to load the snapshot values as actual values for the current data block.

Program Code

```
Code example for loading the snapshot values as actual values
Copies the following program code to clipboard.
```

```
PlcBlock block1 =
PlcSoftware.BlockGroup.Blocks.Find("DataBlock_1");
//Getting the value service for the data block
ValueService valueService =
block1.Interface.GetService<ValueService>();
//Loading the snapshot data as actual values to the PLC
//the PLC must be in 'Online' mode
valueService.LoadSnapshotAsActualValues();
```

Loading start values as actual values

You can load the start values as actual values for a data block via the Openness API. The stored start values of an existing DataBlock can be loaded as actual values with the following workflow:

- Using navigator 'Interface' on a DataBlock request the service 'ValueService'.
- If not yet done before, set specific start values for the DataBlock.
- Call the 'LoadStartValuesAsActualValues' action on the value service to load the start values as actual values for the current data block.

Program Code

```
Code example for loading the start values as actual values
Copies the following program code to clipboard.

PlcBlock block1 =
PlcSoftware.BlockGroup.Blocks.Find("DataBlock_1");
//Getting the value service for the data block
ValueService valueService =
block1.Interface.GetService<ValueService>();
//Loading the start values as actual values to the PLC
//the PLC must be in 'Online' mode
valueService.LoadStartValuesAsActualValues();
```

1.11 Creating and Renaming group for blocks

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
  See Connecting to the TIA Portal
- A project is open.
  See Opening a project

Program code for creating a new block

Modify the following program code to create a group for blocks:

```
Copies the following program code to clipboard.
private static void CreateBlockGroup(PlcSoftware plcsoftware)
//Creates a block group
{
    PlcBlockSystemGroup systemGroup = plcsoftware.BlockGroup;
    PlcBlockUserGroupComposition groupComposition =
systemGroup.Groups;
    PlcBlockUserGroup myCreatedGroup =
groupComposition.Create("MySubGroupName");
}
```

Program code for renaming a new bloack

Modify the following program code to rename a group for blocks:

```
Copies the following program code to clipboard.
private static void RenameBlockGroup(PlcSoftware plcsoftware)
{
PlcBlockSystemGroup systemGroup = plcsoftware.BlockGroup;
PlcBlockUserGroupComposition groupComposition =
systemGroup.Groups;
PlcBlockUserGroup myCreatedGroup =
groupComposition.Create("MySubGroupName");
myCreatedGroup.SetAttribute("Name", "New_name_of_group");
// Or myCreatedGroup.Name = "New_name_of_group";
}
```

See also

Importing configuration data

1.12 Deleting group for blocks

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
  See Connecting to the TIA Portal
- A project is open.
  See Opening a project
- PLC is not online.

Program code

Modify the following program code to delete a group for blocks:



```
Copies the following program code to clipboard.

// Deletes user groups from PlcBlockSystemGroup or
PlcBlockUserGroup

private static void DeleteBlockFolder(PlcSoftware
plcSoftware)

{

    PlcBlockUserGroup group =
plcSoftware.BlockGroup.Groups.Find("myGroup");

    //PlcBlockSystemGroup group = plcSoftware.BlockGroup;

    PlcBlockUserGroupComposition subgroups = group.Groups;

    PlcBlockUserGroup subgroup =
subgroups.Find("myUserGroup");

    if (subgroup != null)

    {

        subgroup.Delete();

    }

}
```

See also

Importing configuration data

1.13 Accessing attributes of all blocks

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
  See Connecting to the TIA Portal

- A project is open.
  See Opening a project

Introduction

You can use the TIA Portal Openness to set the attributes applicable for all blocks using SetAttribute() and SetAttributes() method.

The SetAttributes() can operate on all objects types where writable openness attribute is available. With one input item it behaves as SetAttribute.

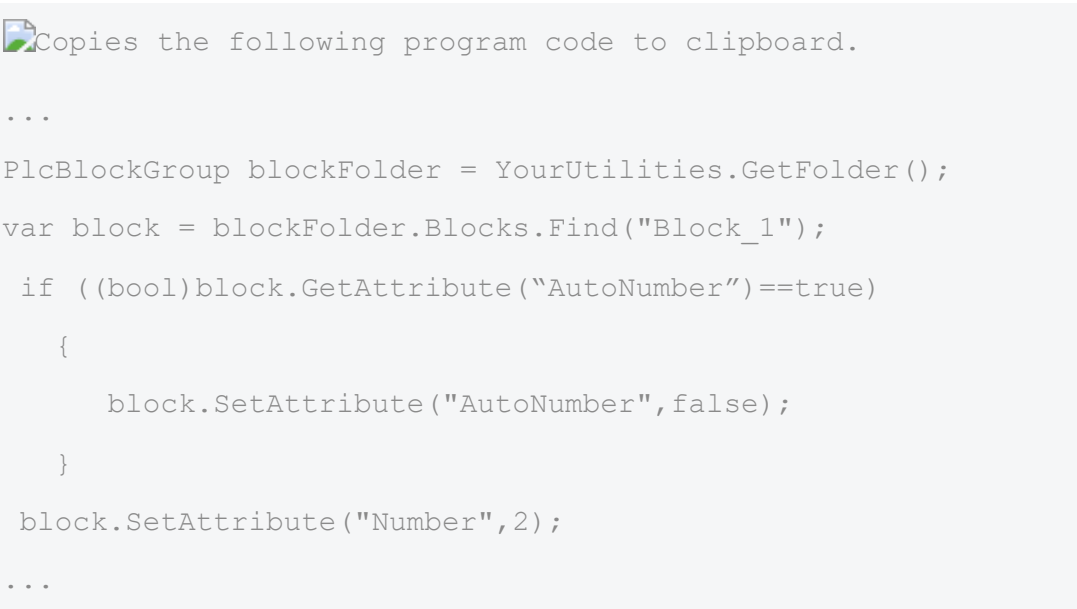
Parameter checking:

- Initial checks are done before the first item is being processed by global PE infrastructure: checking of input items for identical and valid attribute's name, data type mismatch. The same attribute cannot be set twice by one SetAttributes
- Sequential check happens during running of SetAttributes. The order of the processing is the same as the order of the input list elements

The SetAttributes operates in only one transaction. If any attribute of input list cannot be set, the values of attributes are remaining unchanged. SetAttributes sets attributes in the same order as they can be found in the parameter list. The parameters can be dependent or independent of each other. If one parameter depends on one of the previous parameters in the list, this parameter is checked and evaluated according to the current value of the other attribute (not to the original value at the time of calling). If the SetAttributes cannot be executed, details of the error are indicated. The error message contains the first attribute name and its value which could not be managed to set and its reason.

The following program code examples were given based on the two attributes AutoNumber and Number using SetAttribute() (Refer Exporting blocks for all applicable attributes of blocks).

Program code: SetAttribute



Program code: SetAttributes

 Copies the following program code to clipboard.

```
PlcBlock block = SelectBlock("MC-Servo");
if (block != null)
{
IList<KeyValuePair<string, object>> list = new
List<KeyValuePair<string, object>>()
{
new KeyValuePair<string, object>("DataExchangeMode",
OBDataExchangeMode.Synchronous),new KeyValuePair<string,
object>("SynchronousApplicationCycleTime", (float)69)
};
 try
  {
     block.SetAttributes(list);
  }
 catch (EngineeringException e)
  {
    Console.WriteLine("Exception: " + e.Message);
  }
}
```

Read and write access to attributes of PLC blocks

Header Attributes

The following Header attributes are read write accessible for PLC blocks.

- HeaderAuthor (GUI: Author)
- HeaderFamily (GUI: Family)
- HeaderName (GUI: User-defined ID)
- HeaderVersion (GUI: Version)

Code example for setting the ‘Header’ attributes

 Copies the following program code to clipboard.

```
PlcBlock block1 =
PlcSoftware.BlockGroup.Blocks.Find("Block_1");
block1.HeaderAuthor = "NewAuthor";
block1.HeaderFamily = "HeaderFamily";
block1.HeaderName = "HeaderName";
block1.HeaderVersion = new Version(1, 0, 0)
```

Memory Attributes

The following Memory attributes are read only accessible for PLC blocks.

- Load memory (length in bytes) (read-only)
- Work memory (length in bytes) (read-only)

```
Code example for reading the memory attributes
 Copies the following program code to clipboard.

PlcBlock block1 =
PlcSoftware.BlockGroup.Blocks.Find("Block_1");
Console.WriteLine($"LoadMemory: {block1.LoadMemoryLength}");
Console.WriteLine($"WorkMemory: {block1.WorkMemoryLength}");
```

Other block attributes

The following attribute specifying the date of the last download date of the block is available as read-only attribute.

- DownloadDate (read-only)

The following attributes are realized as dynamic attributes, because they are not available for all block entities:

- Available for OB, FB, FC
  - IsIECCheckEnabled
  - SetENOAutomatically
- Available for DB, IDBo
  - IsOnlyStoredInLoadMemory
  - IsWriteProtectedInAS
  - DBAccessibleFromOPCUA (also for TO)

```
Code example for reading the DownloadDate
 Copies the following program code to clipboard.

PlcBlock block1 =
PlcSoftware.BlockGroup.Blocks.Find("Block_1");
Console.WriteLine($"DownloadDate: {block1.DownloadDate}");
```

```
Code example for reading the DownloadDate
Copies the following program code to clipboard.

PlcBlock block1 =
PlcSoftware.BlockGroup.Blocks.Find("Block_1");
var attributeName = "SetENOAutomatically";
//read and display the current value
Console.WriteLine($"SetENOAutomatically:
{block1.GetAttribute(attributeName)}");
//set a new value
block1.GetAttribute(attributeName, true);
// display the new value
Console.WriteLine($"SetENOAutomatically:
{block1.GetAttribute(attributeName)}");
```

Multilingual Title and comments of PLC object

The multilingual title and comment is available for the following openness objects.

- PlcType
- PlcBlock
- TechnologicalInstanceDB

For these objects the 'Name' and 'Title' property return a 'MultilingualText' object which can be used in conjunction with the culture info get or set a culture specific value.The workflow of getting and setting the multilingual values is as follows:

- Find the block or type supporting multilingual comments
- Find the desired 'Language' in the cultures of the project
- Get the 'MultilingualText' object from the available property
- Find the 'MultilingualTextItem' for the desired language
- Get or set the value in the scope of the language

```
Code example for getting and setting multlingual values
Copies the following program code to clipboard.

PlcBlock block1 =
PlcSoftware.BlockGroup.Blocks.Find("Block_1");
//get the language for the multilingual text
CultureInfo cultureInfo = new CultureInfo("de-DE");
Language language =
OpnsProject.LanguageSettings.Languages.Find(cultureInfo);
//get multilingual text property
MultilingualText title = block1.Title;
//get the text item for the specific language
```

```
MultilingualTextItem titleItem = title.Items.Find(language);
// display the current value
Console.WriteLine($"Title in DE: {titleItem.Text}");
// set a new localized value
titleItem.Text = "New value in DE: Neu";
```

1.14 Creating a ProDiag-FB

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
  See Connecting to the TIA Portal
- A project is open.
  See Opening a project

Introduction

You can use the PLCBlock composition's create action to create ProDiag FB with the following parameters.

1. Name
2. Auto number flag
3. Number
   - If "auto number flag" is true, the given block number will be set in case it is free, otherwise a new block will be generated
   - If "auto number flag" is false, the given block number will be set to the block
4. Programming language
   - Invoke create action with ProDiag programming language, then a new FB will be created without IDB.
   - Invoke create action with IDB of ProDiag, than IDB of ProDiag will be created.
   - In any other not supported case, a recoverable exception is thrown.

Program code: Creating a ProDiag-FB

```
Copies the following program code to clipboard.

PlcSoftware plc = ...;
PlcBlockGroup blockFolder = plc.BlockGroup;
PlcBlockComposition blockComposition = blockFolder.Blocks;
if (blockComposition != null)
{
```

```
string fbName = "ProDiag_Block";
bool isAutoNumber = true;
int number = 1;
var progLang = ProgrammingLanguage.ProDiag;
FB block = blockComposition.CreateFB(fbName, isAutoNumber,
number, progLang);
string iDBName="ProDiag_IDB";
string instanceOfName = fbName;
InstanceDB iDbBlock =
blockComposition.CreateInstanceDB(iDBName, isAutoNumber,
number, instanceOfName);
```

See also

Accessing supervisions and properties of ProDiag-FB

1.15 Accessing supervisions and properties of ProDiag-FB

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
  See Connecting to the TIA Portal
- A project is open.
  See Opening a project

Accessing supervisions of User-FB

You can use the TIA Portal Openness to access the supervisions at FB block. Every FB block has the list of supervisions including Classic and S7-1200/1500 PLCs. You can set AssignedProDiagFB at

InstanceDB via the attribute AssignedProDiagFB (Refer Exporting blocks).

You can use the GetAttribute(), GetAttributes(), and SetAttribute() method for accessing the attributes.

Restriction

You cannot use the SetAttributes() in the TIA Portal Openness to set more than one attribute. You will get an exception in TIA Portal Openness for trying to set more than one attribute using SetAttributes().

Program code: Accessing supervisions of ProDiag-FB

```
Copies the following program code to clipboard.
…
PlcBlock iDB = plc.BlockGroup.Blocks.Find("FB_Block_DB");
string fbName =
iDB.GetAttribute("InstanceOfName").ToString();
FB fb = (FB)plc.BlockGroup.Blocks.Find(fbName);
if (fb.Supervisions.Count > 0)Console.WriteLine("Contains
supervisions");
else
Console.WriteLine("Does not contains supervisions");
…
```

Program code: Getting and setting the assigned ProDiag-FB at and IDB

```
Copies the following program code to clipboard.
...
 PlcBlockGroup blockFolder = plc.BlockGroup;
 PlcBlock instanceDB = blockFolder.Blocks.Find("IDB");
 PlcBlock plcProdiag =
blockFolder.Blocks.Find("block_Prodiag");
 instanceDB.SetAttribute("AssignedProDiagFB",
plcProdiag.Name);
 var assignedProDiagFB =
instanceDB.GetAttribute("AssignedProDiagFB");
...
```

See also

Creating a ProDiag-FB

1.16 Assigning ProDiag FB for DBs & Tag Table

Requirement

- The TIA Portal Openness application is connected to the TIA Portal
  See Connecting to the TIA Portal

- A project is open
  See Opening a Project

Application

You can use the TIA Portal Openness to get and set the assigned proDiagFB value of a member in Global DB and TagTable using GetAttribute/SetAttribute method.

The Siemens.Engineering.SW.Blocks.Interface.Member type name provides the following properties:

Property Name	Data type	Access
Name	String	Read
AssignedProDiagFB	String	Read/Write

Program code: Getting proDiagFB Value from DB member & Tag member

Modify the following program code to get prodiagFB in different instances of UDT irrespective of whether the UDT has supervision or not:



```
Copies the following program code to clipboard.
//Simple UDT instance
MemberComposition gdbMembers = dataBlock.Interface.Members;
Member member = gdbMembers.Find("SimpleUdtInstanceMember");
object proDiagFB = member.GetAttribute("AssignedProDiagFB");
//Array of UDT Instance
Member member =
dataBlock.Interface.Members.Find("ArrayUdtMember.ArrayUdtMember[0]");
object proDiagFB = member.GetAttribute("AssignedProDiagFB");
//UDT instance inside Struct
Member member =
dataBlock.Interface.Members.Find("structMember.UdtInstanceMember");
object proDiagFB = member.GetAttribute("AssignedProDiagFB");
//UDT Instance inside Array of Struct
MemberComposition gdbMembers = dataBlock.Interface.Members;
```

```
Member member =
gdbMembers.Find("ArrayOfStructMember.ArrayOfStructMember[0].UdtInstanceMember"
object proDiagFB = member.GetAttribute("AssignedProDiagFB");
```

Modify the following program code to get prodiagFB in tagTable:



```
Copies the following program code to clipboard.
// UDT Instance in TagTable
PlcTagTable tagTable = plc.TagTableGroup.TagTables.Find("Tag
table_1");
PlcTag tag = tagTable.Tags.Find("Tag_1");
object proDiagFB = tag.GetAttribute("AssignedProDiagFB ");
```

Program code: Setting proDiagFB Value to DB & Tag member

Modify the following program code to set assigned ProdiagFB attribute to a member in DB:



```
Copies the following program code to clipboard.
//Simple UDT instance
MemberComposition gdbMembers = dataBlock.Interface.Members;
Member member = dataBlock.Interface.Members.Find("SimpleUdtInstanceMember");
member.SetAttribute("AssignedProDiagFB", "ProDiagFBName");
//Array of UDT Instance
Member member = dataBlock.Interface.Members.Find("ArrayUdtMember.ArrayUdtMembe
member.SetAttribute("AssignedProDiagFB", "ProDiagFBName");
//UDT instance inside Struct
Member member = dataBlock.Interface.Members.Find("structMember.UdtInstanceMemb
member.SetAttribute("AssignedProDiagFB", "ProDiagFBName");
//UDT Instance inside Array of Struct
Member member =
dataBlock.Interface.Members.Find("ArrayOfStructMember.ArrayOfStructMember[0].U
member.SetAttribute("AssignedProDiagFB", "ProDiagFBName");
```

Modify the following program code to set the assigned ProdiagFB attribute to a tag member:



```
Copies the following program code to clipboard.
// UDT Instance in TagTable
```

```
var tagTable = plc.TagTableGroup.TagTables.Find("Tag table_1");
var tag = tagTable.Tags.Find("Tag_1");
tag.SetAttribute ("AssignedProDiagFB ", "ProDiagFBName");
```

Note

The GetAttribute and SetAttribute of ProDiagFB is a dynamic attribute and valid only for UDT instance in Global Db and Tag Table.

Objects like simple datatype for example int, string etc and derived datatype like struct, array, all system defined types , the user constants, and system constants types do not support this attribute.

Exception will be thrown if you try to get/set the attribute for an object which are not valid.

See also

Connecting to the TIA Portal
Opening a project

1.17 Reading ProDiag-FB blocks and attributes

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
  See Connecting to the TIA Portal
- You have opened a project via a TIA Portal Openness application
  See Opening a project

Application

You can use the TIA Portal Openness to read the ProDiag function block version, and other ProDiag related attribute values. You can use GetAttribute ( ) and GetAttributes ( ) methods to read the ProDiag FBs language specific attributes present.

Attributes

The following attributes are supported by ProDiag-FB in Openness:

Attributes	Type
ProDiagVersion	Version

InitialValueAcquisition	bool
UseCentralTimeStamp	bool

See also

Connecting to the TIA Portal
Opening a project

1.18 Exporting ProDiag alarm message

Requirement

- The TIA Portal Openness is connected to the TIA Portal
  See Connecting to the TIA Portal
- A project is open
  See Opening a project

Application

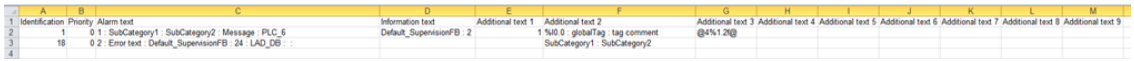
You can use the ExportProDIAGInfo() on CodeBlock in the TIA Portal Openness to export alarm message for a specific ProDiag block in csv format. The resulting output can be opened in microsoft excel.

To export the ProDiag block in csv format, you make sure that the programming language of a ProDiag-FB is "ProDiag". Otherwise, a recoverable exception will be thrown to you.

Export file

The content of the ProDiag exported file contains the identification consisting of the client alarm ID and the supervision ID in hexadecimal format. The second column includes the priority of the supervision category.

An example for the content of an exported file is shown below.



	A	B	C	D	E	F	G	H	I	J	K	L	M
1	Identification	Priority	Alarm text	Information text	Additional text 1	Additional text 2	Additional text 3	Additional text 4	Additional text 5	Additional text 6	Additional text 7	Additional text 8	Additional text 9
2	1	0	1 : SubCategory1 : SubCategory2 : Message : PLC_6	Default_SupervisionFB : 2	1	%I0.0 : globalTag : tag comment	@4%1.2I@						
3	18	0	2 : Error text : Default_SupervisionFB : 24 : LAD_DB : :			SubCategory1 : SubCategory2							
4													

Program code

```
Copies the following program code to clipboard.

private static void ExportProDiagBlock(PlcBlock plcBlock)

{

FB functionBlock = plcBlock as FB;
```

```
if (functionBlock != null &&
functionBlock.ProgrammingLanguage ==
ProgrammingLanguage.ProDiag && functionBlock.IsConsistent)
{
// Define absolute directory path where the CSV files will be
stored
string directoryPath = Path.Combine(@"D:\Temp\ProDiagInfos",
functionBlock.Name);
DirectoryInfo directoryInfo = new
DirectoryInfo(directoryPath);
// Export the alarm messages of a ProDiag block in CSV format
functionBlock.ExportProDIAGInfo(directoryInfo);
}
}
```

1.19 Adding an external file

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.

  See Connecting to the TIA Portal
- You have opened a project via a TIA Portal Openness application:

  See Opening a project

Application

You can add an external file to a PLC. This external file is stored in the file system under the defined path.

The following formats are supported:

- STL
- SCL
- DB
- UDT

Note

Accessing groups in the "External source files" folder is not supported.

An exception is thrown if you specify a file extension other than *.AWL, *.SCL, *.DB or *.UDT.

Program code

Modify the following program code to create an external file in the "External source files" folder from a block.

```
Copies the following program code to clipboard.

private static void CreateBlockFromFile(PlcSoftware plcSoftware)
// Creates a block from a AWL, SCL, DB or UDT file
{
    PlcExternalSource externalSource =
plcSoftware.ExternalSourceGroup.ExternalSources.CreateFromFile("SomeBloc
}
```

1.20 Generate source from block

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
  See Connecting to the TIA Portal
- A project is open.
  See Opening a project
- PLC is not online.

Introduction

The TIA Portal Openness API supports the generation of sources in UTF-8 from STL or SCL blocks, data blocks and PLC data types (user data types). To generate a source file of a block, invoke the method `GenerateSource` on the `PlcExternalSourceSystemGroup` instance.

The file encoding for sources with special characters in UTF-8 must be UTF-8 with BOM (Byte Order Mark). While typically the file encoding is recognized automatically for read operations, the

encoding must be specified explicitly for write operations.

The below example shows how to write a content into an external source file in UTF-8 with BOM:

```
Copies the following program code to clipboard.

string fileContent = "// SCL source code ...";
```

```
System.IO.File.WriteAllText("D:\\Sources\\Source_1.scl",
fileContent, new System.Text.UTF8Encoding(true));
```

The scale of the generated source file depends on the generation option of this function:

- GenerateOptions.None

  Generate source from provided blocks only.

- GenerateOptions.WithDependencies

  Generate source including all dependent objects.

The interface `Siemens.Engineering.SW.ExternalSources.IGenerateSource` indicates that a source can be generated.

Only the STL and SCL programming languages are supported for blocks. Exceptions are thrown in the following cases:

- Programming language is not STL or SCL
- A file of the same name already exists at the target location

Only the "*.udt" file extension is supported for user data types. Exceptions are thrown in the following cases:

- The file extension is not "*.db" for DBs
- The file extension is not "*.awl" for STL blocks
- The file extension is not "*.scl" for SCL blocks

Program code

Modify the following program code to generate source files from blocks and types:

```
Copies the following program code to clipboard.
PlcExternalSourceSystemGroup.GenerateSource(IEnumerable<IGenerateSource>
plcBlocks, FileInfo sourceFile, GenerateOptions generateOptions);
// ...
//examples
// ...
var blocks = new List<PlcBlock>(){block1};
var fileInfo = new FileInfo(@"C:\temp\SomePathHere.scl");
PlcExternalSourceSystemGroup systemGroup = ...;
systemGroup.GenerateSource(blocks, fileInfo,
GenerateOptions.WithDependencies);
// exports all blocks and with all their dependencies(e.g. called
blocks, used DBs or UDTs)
```

```
// as ASCII text into the provided source file.
// ...
// or
// ...
var types = new List<PlcType>(){udt1};
var fileInfo = new FileInfo(@"C:\temp\SomePathHere.udt");
PlcExternalSourceSystemGroup systemGroup = ...;
systemGroup.GenerateSource(types, fileInfo,
GenerateOptions.WithDependencies );
// exports all data types and their used data types into the
provided source file.
// ...
```

See also

Importing configuration data

1.21 Generating blocks from source

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.

  See Connecting to the TIA Portal
- A project is open.

  See Opening a project
- PLC is not online.

Application

You can generate blocks from all external files in the "External source files" group. Only external files with the format ASCII are supported.

Note

Existing blocks are overwritten.

An Exception is thrown if an error occurs during the calling. Before TIA Portal Openness V18, The first 256 characters of each error message are contained in the notification of the Exception. With TIA Portal Openness V18, the first 4096 characters of the message are returned.

The project is reset to the processing state prior to the execution of the `GenerateBlocksFromSource` method.

Program code

Modify the following program code to generate blocks from all external files in the "External source files" group.

```
Copies the following program code to clipboard.
// Creates a block from an external source file
PlcSoftware plcSoftware = ...;
foreach (PlcExternalSource plcExternalSource in
plcSoftware.ExternalSourceGroup.ExternalSources)
{
plcExternalSource.GenerateBlocksFromSource();
}
```

1.22 Generating from source of known source format

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
  See Connecting to the TIA Portal
- A project is open.

  See Opening a project
- PLC is not online

Application

The TIA Portal Openness API interface supports the generation of block and UDT object. There are conditions where the generated block and UDT are failed to compile.

Typical missing or invalid parts which makes the block and UDT not compilable:

- Reference to a not existing type
- Missing symbol used

The GenerateBlocksFromSource() fails to generate UDT and block with following result:

- A recoverable exception is thrown
- The error message(s) of the generating to the exception’s message
- Deletes the generated block/UDT

A new overload of GenerateBlocksFromSource() invoke with GenerateBlockOptions.KeepOnError will lead to the below result:

- Recoverable exception will not be thrown regardless the generating results
- No information about the generation result of any generated block(s)
- Returns with list of IEngineeringObjects of the the generated blocks (with or without generation errors)

If the GenerateBlocksFromSource() invoke with GenerateBlockOptions.None and then generation of block and UDT is error free with following result:

- Recoverable exception will be thrown for failed generation
- Includes the error message(s) of the generating to the exception's message
- Deletes the generated block/UDT

In case of flawless generation, it

- doesn't throw any exception
- doesn't provide information about the generation result of any generated blocks
- returns with an IList of IEngineeringObjects of the the generated blocks
  - this is the same behavior as having called it with GenerateBlockOptions.KeepOnError.

Program code



```
Copies the following program code to clipboard.
try
{
  IList<IEngineeringObject> generatedObjects =
externalSource.GenerateBlocksFromSource(GenerateBlockOptions.KeepOnErro
  foreach (IEngineeringObject engineeringObject in generatedObjects)
{
  CompilerResult compilerResult = null;
  string objectName = null;
  if (engineeringObject is PlcBlock)
  {
   // handle case for PLcBlock
   // e.g. retrieve the compiler result
   PlcBlock block = (PlcBlock)engineeringObject;
   objectName = block.Name;
```

```
   compilerResult = block.Compile();
  }
  else if (engineeringObject is PlcType)
  {
   // handle case for PlcType
   // e.g. retrieve the compiler result
   PlcType plcType = (PlcType)engineeringObject;
   objectName = plcType.Name;
   compilerResult = plcType.Compile();
  }
  // handle the compiler result
  if (compilerResult != null)
  {
   if (compilerResult.State == CompilerResultState.Error)
   {
    Console.WriteLine("Object '{0}' could not be compiled
successfully!", objectName);
    Console.WriteLine("Number of compiler errors: {0}",
compilerResult.ErrorCount);
    foreach (CompilerResultMessage compilerResultMessage in
compilerResult.Messages)
    {
     Console.WriteLine(compilerResultMessage.Description);
    }
   }
   else
   {
    Console.WriteLine("Object '{0}' could be compiled
successfully.", objectName);
   }
  }
 }
```

```
}
catch (RecoverableException exception)
{
  // handle recoverable exception
  Console.WriteLine(exception.Message);
}
```

See also

Connecting to the TIA Portal
Opening a project

1.23 Deleting user data type

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
  See Connecting to the TIA Portal
- A project is open.
  See Opening a project
- PLC is not online.

Program code

Modify the following program code to delete a user type:



```
Copies the following program code to clipboard.
private static void DeleteUserDataType(PlcSoftware
plcSoftware)
{
    PlcTypeSystemGroup typeGroup = plcSoftware.TypeGroup;
    PlcTypeComposition dataTypes = typeGroup.Types;
    PlcType dataType = dataTypes.Find("DataTypeName");
    if (dataType != null)
    {
        dataType.Delete();
    }
}
```

See also

Importing configuration data

1.24 Accessing PLC system data types

Requirement

- The TIA Portal Openness application is connected to the TIA Portal
  See Connecting to the TIA Portal
- A project is open

  See Opening a Project

Introduction

You can use the properties Siemens.Engineering.SW.PlcSystemTypeGroup and Siemens.Engineering.SW.PlcSystemTypeGroupComposition to access a list of system-generated types.

The PlcSystemTypeGroupComposition provides access to the PLC system type groups, while the PlcSystemTypeGroup allows you to access each system's data types.

Program code

Modify the following program code to access the system data types:



```
Copies the following program code to clipboard.

private static void
accessPlcSystemTypeGroup(PlcSystemTypeGroupComposition,
m_target)
{
// SystemTypeGroups is a composition of system data types
PlcSystemTypeGroup plcSystemTypeGroup =
m_target.TypeGroup.SystemTypeGroups.FirstOrDefault();
}
```

See also

Opening a project

1.25 Deleting an external file

Requirement

- The TIA Portal Openness application is connected to the TIA Portal
  see Connecting to the TIA Portal
- You have opened a project via a TIA Portal Openness application:
  see Opening a project
- PLC is not online

Program code

Modify the following program code to delete an external file in the "External source files" group.

Note

Access to groups in the "External source files" group is not supported.

```
// Deletes an external source file
private static void DeleteExternalSource(PlcSoftware plcSoftware)
{
    PlcExternalSource externalSource =
plcSoftware.ExternalSourceGroup.ExternalSources.Find("myExternalsource")
    externalSource.Delete();
}
```

1.26 Starting the block editor

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
  See Connecting to the TIA Portal
- A project is open.
  See Opening a project
- Instance of the TIA Portal is opened with user interface.

Program code

Modify the following program code to start the associated editor for an object reference of the type `PlcBlock` in the TIA Portal instance:

```
//Opens a block in a block editor
private static void StartBlockEditor(PlcSoftware plcSoftware)
{
    PlcBlock plcBlock =
plcSoftware.BlockGroup.Blocks.Find("MyBlock");
    plcBlock.ShowInEditor();
}
```

Modify the following program code to open the associated editor for an object reference of the type `PlcType` in the TIA Portal instance:



```
Copies the following program code to clipboard.
//Opens a udt in udt editor
private static void StartPlcTypEditor(PlcSoftware
plcSoftware)
{
    PlcTypeComposition types = plcSoftware.TypeGroup.Types;
    PlcType udt = types.Find("my_udt");
    udt.ShowInEditor();
}
```

See also

Importing configuration data

1.27 Changing blocks using fingerprints

Requirement

- The application is connected to the TIA Portal using TIA Portal Openness
  See Connecting to the TIA Portal
- A project is open
  See Opening a project

Introduction

The TIA Portal Openness supports fingerprint with name Program Code which compares pure code and ignores inline comments, multi line comments, line breaks and Tabs. The Openness fingerprintId ProgramCode is used to extract the checksum value of the fingerprint “Program Code”

You can use the TIA Portal Openness to detect changes inside of blocks and UDT. You can achieve the functionality by comparing the fingerprints of the object. A fingerprint instance contains a FingerprintId which defines the fingerprint type and the fingerprint value as a string.

All provided fingerprints consider only user input, no compilation result, or any other change made by the system. You can use the FingerprintProvider service to retrieve the instance of fingerprints. The FingerprintProvider service is available for blocks and UDTs, but not for tag tables. The GetFingerprints() is invoked on the fingerprints instance to calculate and return all available fingerprints object. The block or UDT needs to be consistent before calling fingerprints to get the valid fingerprints. Otherwise, a RecoverableException is thrown. When a fingerprint is still invalid after its calculation, a RecoverableException is thrown.

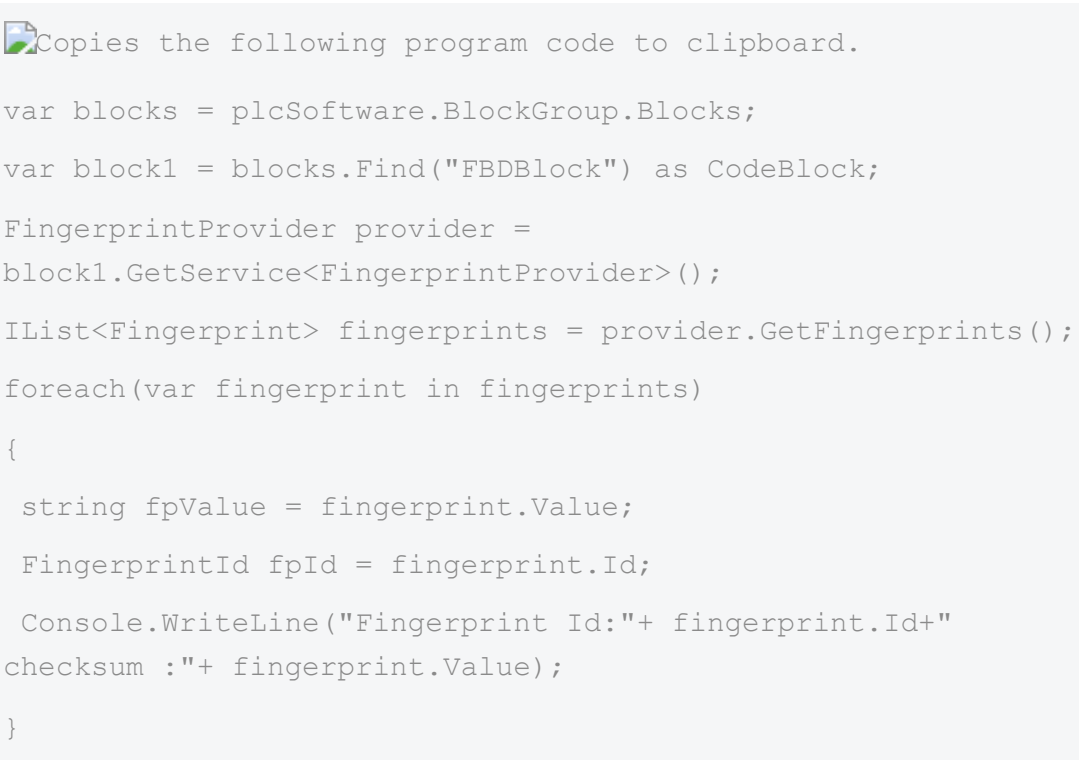
The enumeration FingerprintId lists all kind of fingerprints supported in Openness:

Value	Description
Code	Considers all changes in the code inside the body of the block. It does not consider the compilation result.
Interface	Considers all changes in the interface of a block. Including start values of a DB
Properties	Considers changes in the properties of a block. e.g. name, number
Comments	Considers changes in the comments of a block. In case of OBs the fingerprint also changes when the list of available languages in project language setting changes
LibraryType	Exists when a block is connected to a library type
Texts	With V15 SP1 this fingerprint only exists for Graph blocks
Alarms	Exists when a block uses alarming.
Supervision	Exists when a block contains supervision
TechnologyObject	Exists only for technology object DBs
Events	Exists only for OB

TextualInterface	Exists when the block has a textual interface
ProgramCode	Considers pure code changes and ignores inline comments, multi-line comments, line breaks and tabs

Program code

Modify the following program code to extract the checksum value of the fingerprint “Program Code":



```
Copies the following program code to clipboard.
var blocks = plcSoftware.BlockGroup.Blocks;
var block1 = blocks.Find("FBDBlock") as CodeBlock;
FingerprintProvider provider =
block1.GetService<FingerprintProvider>();
IList<Fingerprint> fingerprints = provider.GetFingerprints();
foreach(var fingerprint in fingerprints)
{
 string fpValue = fingerprint.Value;
 FingerprintId fpId = fingerprint.Id;
 Console.WriteLine("Fingerprint Id:"+ fingerprint.Id+"
checksum :"+ fingerprint.Value);
}
```

See also

Connecting to the TIA Portal
Opening a project

1.28 Generating/deleting blocks for user defined pages

Requirement

- The TIA Portal Openness application is connected to the TIA Portal
  See Connecting to the TIA Portal
- A project is open
  See Opening a Project

Application

You can use the TIA Portal Openness to create user defined web pages within the web server of PLCs. The data for these user defined pages are stored in specific system generated data blocks of the PLC. You can also edit and delete these blocks manually. But if you modify any data or properties in these data blocks the web server will not function correctly.

In TIA Portal Openness, the new service WebserverUserDefinedPages is added at the DeviceItem PLC. You can use this service to generate the DBs for user defined pages: List<PLCBlock> GenerateBlocks(Arguments). When this function is called the corresponding DBs are generated and a list of generated DBs is returned to user.

There are two possible overloads for this function:

- IList<PlcBlock> WebserverUserDefinedPages.GenerateBlocks(WebDBGenerateOptions GenerateOptions);
- IList<PlcBlock> WebserverUserDefinedPages.GenerateBlocks(System.IO.DirectoryInfo htmlDirectory, System.IO.FileInfo defaultHTMLPage, string applicationName, WebDBGenerateOptionsGenerateOptions);

The parameter values for HTML directory, HTML page and application name are not stored in the project data. So the values of the corresponding dynamic attributes are not changed.

			attribute WebserverDefaultHTMLPage
applicationName	string	Optional	Specifies the application name independently from the value of the dynamic attribute WebserverApplicationName

The GenerateBlocks() will throw recoverable exception:

- If web server is disabled (WebserverActive == False) - "Webserver has to be enabled"
- If WebserverHTMLDirectory is empty or the path is invalid - "HTML directory has invalid path"
- If WebserverdefaultHTMLPage is empty or the path is invalid - "Default HTML page has invalid path"
- When the WebserverHTMLDirectory is too long - "HTML directory has too large data"
- The Application Name is invalid - "Application Name is invalid"
- The Dynamic Content is invalid - "Dynamic Content is invalid"
- The Control DB number is invalid - "Control DB number is invalid"
- The DB Start Number is invalid - "DB Start Number is invalid"
- Blocks are already present: If the user tries to generate a block with a name which already exists- ""Delete existing blocks before generating new blocks" (Only thrown if WebDBGenerateOptions.None is used)

There is no specific function to delete all blocks connected to the user defined pages. You have to manually delete the blocks that are handed back to them by the generate action or navigate to the corresponding DBs as it is already possible in TIA Portal Openness.

Program code

Modify the following program code to generate a block for user defined page:

```
Copies the following program code to clipboard.

DirectoryInfo htmlDirectory = ...;

FileInfo defaultHTMLPage = ...;

string applicationName = ...;

DeviceItem deviceItem = ...;

var WebserverUserDefinedPagesService =
deviceItem.GetService<WebserverUserDefinedPages>();

IList<PlcBlock> block =
WebserverUserDefinedPagesService.GenerateBlocks(WebDBGenerateOptions.Nor
```

```
IList<PlcBlock> blocks =
WebserverUserDefinedPagesService.GenerateBlocks(htmlDirectory,
defaultHTMLPage, applicationName,
WebDBGenerateOptions.Override);
```

See also

Connecting to the TIA Portal
Opening a project

1.29 Writing access for OB block priority attribute

Requirement

- The TIA Portal Openness application is connected to the TIA Portal
  See Connecting to the TIA Portal
- A project is open
  See Opening a project

Application

You can use the TIA Portal Openness to provide write access for block property especially for OBs. You can change the priority of a block both in unit and non-unt programming model using TIA Portal Openness.

The following property is available to access for block:

Property name	Data type	Access
PriorityNumber	Integer	Read/Write

Program code

Modify the following program code to set the priority for OB block:

```
 Copies the following program code to clipboard.

PlcSoftware plcSoftware = ...;

PlcBlock obBlock =
plcSoftware.BlockGroup.Blocks.Find("CyclicInterrupt");

obBlock.SetAttribute("PriorityNumber", 14);
```

See also

Opening a project

1.30 Generating block/UDT from external source file in specific user group

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
  See Connecting to the TIA Portal
- You have opened a project via a TIA Portal Openness application.
  See Opening a project

Application

You can use the TIA Portal Openness to generate blocks/ udts under user specific groups in both PLC programming model and unit programming model..

Functions

Method name with signature

- Public IList<IEngineeringObject> PlcExternalSource.GenerateBlocksFromSource(PlcBlockUserGroup blockuserGroup, GenerateBlockOption generateBlockOption)
- Public IList<IEngineeringObject> PlcExternalSource.GenerateBlocksFromSource(PlcBlockUserGroup blockuserGroup, GenerateBlockOption generateBlockOption)

Program code:Generation of blocks from source under specific user groups

Modify the following program code to retrieve the user group information under which the blocks to be generated:

```
Copies the following program code to clipboard.

Siemens.Engineering.SW.Blocks.PlcBlockUserGroup folder =
plc.BlockGroup.Groups.Find("Group_1");
```

Modify the following program code to generate blocks under groups without any options.

```
Copies the following program code to clipboard.

plc.ExternalSourceGroup.ExternalSources.Find("Block_1.scl").GenerateBloc
GenerateBlockOption.None);
```

Modify the following program code to generate blocks with Options as KeepOnError:

```
Copies the following program code to clipboard.
```

```
plc.ExternalSourceGroup.ExternalSources.Find("Block_1.scl")
GenerateBlocksFromSource(folder,
GenerateBlockOption.KeepOnError);
```

Program code: Generation of UDTs from source under specific user groups




```
Siemens.Engineering.SW.Types.PlcTypeUserGroup folder1 =
plc.TypeGroup.Groups.Find("Group_1");
```

Modify the following program code to generateUDTs under groups without any Options:




```
plc.ExternalSourceGroup.ExternalSources.Find("User_data_type_1.udt").Ge
GenerateBlockOption.None);
```

Modify the following program code to generate blocks with Options as KeepOnError:




```
plc.ExternalSourceGroup.ExternalSources.Find("User_data_type_1.udt").Ge
KeepOnError);
```

See also

Connecting to the TIA Portal
Opening a project

1.31 Generating loadable file for blocks outside of software units

Requirement

- The TIA Portal Openness application is connected to the TIA Portal
  See Connecting to the TIA Portal
- The TIA Portal project is open
  See Opening a Project
- The PLC must be compile clean

Introduction

You can use the TIA Portal Openness to generate a loadable file for blocks outside of software units via the following method `GenerateLoadable(FileInfo path,`

```
IEnumerable<PlcBlock> blocks, TargetOption targetOption)
```

Note

The generation of a loadable file for blocks is supported for PLC1518 or its subtype whose firmware is V2.8 or higher.

Restriction

The following restrictions apply when selecting the block:

- The block must not be a system block
- The block must not be a PRODIAG block
- The block must not be a failsafe block
- If the block is an OB, it must not be of any other type than ProgramCycle
- The block must not have a dynamic binding
- The block must not be a technology object (TO)

Parameter and Enum

The GenerateLoadable() accepts the following parameters to generate a loadable file for blocks:

Parameter name	Data type	Descri
path	FileInfo	Specif the na and th storag locatio for the loadab file
blocks	System.Collections.Generic.IEnumerable<Siemens.Engineering.SW.Blocks.PlcBlock>	Specif the lis progra blocks outsid softwa units t

		generate the loadable file from	
targetOption	Siemens.Engineering.SW.Loader.TargetOption	Specifies whether the loadable file is to be created for a real PLC or for a simulated PLC	If TargetOption.PlcSim is passed, all blocks must have been compiled with simulation support

The TargetOption parameter uses the following enum values to generate a loadable file:

Value	Description
TargetOption.None	The default value, should not be used.
TargetOption.Plc	To generate a loadable file for a real PLC.
TargetOption.PlcSim	To generate a loadable file for a simulated PLC.

Program code

To generate a loadable file for blocks outside of software units modify the following program code:

```
Copies the following program code to clipboard.

private void GenerateLoadableProgramBlock(string
loadablePath)
{
// Getting an access to the PLC device of the project
PlcSoftware plc = GetPlc();
// The GetService() will return null if the PLC is not
supported.
```

```
var loadableProviderService =
plc.GetService<LoadableProvider>();
// Create a variable file with name "file" to access the
loadable path
var file = new FileInfo(loadablePath);
// Create a variable with name "blocks" for a list of program
blocks
// from the non-software unit program.
var blocks = new List<PlcBlock>
{
// Accessing a specific block
// with name "Block_1" under plc
plc.BlockGroup.Blocks.Find("Block_1") ?? throw new
InvalidOperationException()
};
// The loadableProviderService generates a loadable file for
program block
// by calling the GenerateLoadable() with respective
parameters.
try
{
loadableProviderService.GenerateLoadable(file, blocks,
TargetOption.Plc);
}
catch (EngineeringTargetInvocationException e)
{
// An EngineeringTargetInvocation exception is thrown
// if the generation of the loadable file fails.
// The exception message contains hints on
// what caused the loadable file creation to fail.
Console.WriteLine("Generating the loadable file failed.");
Console.WriteLine(e);
}
```

```
}
```

Exception

The following possible exception message are listed below:

- It is only possible to create a loadable file from a whole software unit, not from a single block inside of a unit.
- The selected blocks are from different PLCs.
- Destination directory not found: '{0}'.
- It is not possible to create a loadable file from failsafe block: '{0}'.
- Destination file already exists: '{0}'.
- The loadable file is not accessible.
- An internal failure occured.
- The PLC has to be compiled.
- Internal serialization of loadable file failed.
- Internal serialization failed, because loadable file was disposed.
- The specified path, file name, or both exceed the system-defined maximum length.
- The PLC '{0}' is not supported.
- The parent PLC of the service and of the target are different.
- Some block(s) cannot be simulated. If a block is a library block, use a library with simulation support. Otherwise, select the option "Support simulation during block compilation" in the project properties and recompile the block.
- It is not possible to create a loadable file from a system block: '{0}'.
- Access to file is denied.
- Software units containing PLC data types are not supported.
- It is not possible to create a loadable file from selected object(s).
- It is not possible to create a loadable file from OB '{0}', because it does not have the event class 'Program cycle.
- It is not possible to create a loadable file from a Prodiag block: '{0}'.
- It is not possible to create a loadable file from selected object: '{0}.
- It is not possible to create a loadable file from block '{0}' with dynamic binding enabled.

See also

Generating loadable file for software units

Tags and Tag tables

2

2.1 Starting the 'PLC Tags' editor

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
  See Connecting to the TIA Portal
- A project is open.
  See Opening a project
- Instance of the TIA Portal is opened with user interface.

Program code

Modify the following program code to start the corresponding editor for an object reference of the type `PlcTagTable` in the TIA Portal instance:

```
 Copies the following program code to clipboard.

//Opens tagtable in editor "Tags"
private static void OpenTagtableInEditor(PlcSoftware
plcSoftware)
{
    PlcTagTable plcTagTable =
plcSoftware.TagTableGroup.TagTables.Find("MyTagTable");
plcTagTable.ShowInEditor();
}
```

See also

Importing configuration data

2.2 Querying system groups for PLC tags

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
  See Connecting to the TIA Portal

- A project is open.
  See Opening a project
- A PlcSoftware instance was retrieved from a PLC device item.
  See Querying PLC and HMI targets

Program code

Modify the following program code to query the system group for PLC tags:

```
Copies the following program code to clipboard.

//Retrieves the plc tag table group from a plc
private PlcTagTableSystemGroup
GetControllerTagfolder(PlcSoftware plcSoftware)
{
    PlcTagTableSystemGroup plcTagTableSystemGroup =
plcSoftware.TagTableGroup;
    return plcTagTableSystemGroup;
}
```

2.3 Creating PLC tag table

Requirements

- The TIA Portal Openness application is connected to the TIA Portal.
  See Connecting to the TIA Portal
- A project is open.
  See Opening a project
- A PlcSoftware instance was retrieved from a PLC device item.
  See Querying PLC and HMI targets

Program code

Modify the following program code to create the PLC tag table. It creates a new tag table with the given name in the composition.

```
Copies the following program code to clipboard.

PlcTagTable myTable =
plc.TagTableGroup.TagTables.Create("myTable");
```

See also

Querying PLC and HMI targets

2.4 Enumerating user-defined groups for PLC tags

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
  See Connecting to the TIA Portal
- A project is open.
  See Opening a project
- A PlcSoftware instance was retrieved from a PLC device item.
  See Querying PLC and HMI targets

Introduction

Subfolders are taken into account recursively for enumeration.

The following attributes are accessed in PLCTagTableUserGroup:

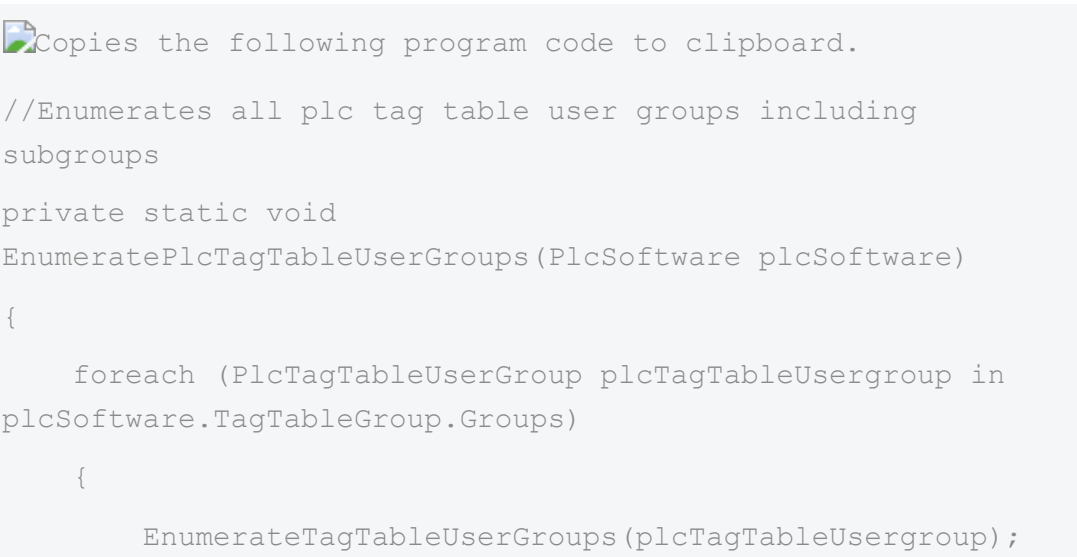
Name	Type	Type
Name[1]	string	Read-only

Note

In TIA Portal Openness version 20 and above, the Name attribute has write access. In versions earlier than 20, the Name attribute does not have write access.

Program code: Enumerating user-defined groups for PLC tags

Modify the following program code to enumerate user-defined groups for PLC tags:



```
    }
}
private static void
EnumerateTagTableUserGroups(PlcTagTableUserGroup
tagTableUsergroup)
{
    foreach (PlcTagTableUserGroup plcTagTableUsergroup in
tagTableUsergroup.Groups)
    {
        EnumerateTagTableUserGroups(plcTagTableUsergroup);
        // recursion
    }
}
```

Program code: Accessing a user-defined group

Modify the following program code to access a user-defined group for PLC tags:

```
 Copies the following program code to clipboard.
//Gives individual access to a specific plc tag table user
folder
private static void
AccessPlcTagTableUserGroupWithFind(PlcSoftware plcSoftware,
string folderToFind)
{
    PlcTagTableUserGroupComposition
plcTagTableUserGroupComposition =
plcSoftware.TagTableGroup.Groups;
    PlcTagTableUserGroup controllerTagUserFolder =
plcTagTableUserGroupComposition.Find(folderToFind);
    // The parameter specifies the name of the user folder
}
```

2.5 Creating user-defined groups for PLC tags

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
  See Connecting to the TIA Portal
- A project is open.
  See Opening a project

Application

The TIA Portal OpennessAPI interface supports the creation of a user-defined group for PLC tags.

Program code

Modify the following program code to create a user-defined group for PLC tags:

```
Copies the following program code to clipboard.
//Creates a plc tag table user group
private static void CreatePlcTagTableUserGroup(PlcSoftware
plcSoftware)
{
    PlcTagTableSystemGroup systemGroup =
plcSoftware.TagTableGroup;
    PlcTagTableUserGroupComposition groupComposition =
systemGroup.Groups;
    PlcTagTableUserGroup myCreatedGroup =
groupComposition.Create("MySubGroupName");
    // Optional;
    // create a subgroup
    PlcTagTableUserGroup mySubCreatedGroup =
myCreatedGroup.Groups.Create("MySubSubGroupName");
}
```

2.6 Deleting user-defined groups for PLC tags

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
  See Connecting to the TIA Portal
- A project is open.
  See Opening a project

Application

The TIA Portal Openness API interface supports the deletion of a specific user-defined group for PLC tag tables.

Program code

Modify the following program code to delete a specific user-defined group for PLC tag tables:

```
Copies the following program code to clipboard.
private static void DeletePlcTagTableUserGroup(PlcSoftware
plcSoftware)
{
    PlcTagTableUserGroup group =
plcSoftware.TagTableGroup.Groups.Find("MySubGroupName");
    if (group != null)
    {
        group.Delete();
    }
}
```

2.7 Enumerating PLC tag tables in a folder

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
  See Connecting to the TIA Portal
- A project is open.
  See Opening a project

Program code: Enumerating PLC tag tables

Modify the following program code to enumerate all PLC tag tables in system groups or in user-defined groups:

```
Copies the following program code to clipboard.
//Enumerates all plc tag tables in a specific system group or
and user group
private static void
EnumerateAllPlcTagTablesInFolder(PlcSoftware plcSoftware)
```

```
{
    PlcTagTableComposition tagTables =
plcSoftware.TagTableGroup.TagTables;
    // alternatively, PlcTagTableComposition tagTables =
plcSoftware.TagTableGroup.Groups.Find("UserGroup
XYZ").TagTables;
    foreach (PlcTagTable tagTable in tagTables)
    {
        // add code here
    }
}
```

Program code: Accessing PLC tag table

Modify the following program code to access the PLC tag table:

```
Copies the following program code to clipboard.
//Gives individual access to a specific Plc tag table
private static void AccessToPlcTagTableWithFind(PlcSoftware
plcSoftware)
{
    PlcTagTableComposition tagTables =
plcSoftware.TagTableGroup.TagTables;
    // alternatively, PlcTagTableComposition tagTables =
plcSoftware.TagTableGroup.Groups.Find("UserGroup
XYZ").TagTables;
    PlcTagTable controllerTagTable = tagTables.Find("Tag
table XYZ");
    // The parameter specifies the name of the tag table
}
```

2.8 Querying information from a PLC tag table

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
  See Connecting to the TIA Portal

- A project is open.
  See Opening a project

Application

Via PLC tag tables you can access user constants, system constants and tags. The count of the tag composition of a tag table is equal to the number of tags in that tag table. The PLCTagTable contains the following navigators, attributes, and actions.

The following attributes are accessed in PLC tag table.

Name	Type	Type
IsDefault	bool	Read-only
ModifiedTimeStamp	DateTime	Read-only
Name[1]	string	Read-only

Note

In TIA Portal Openness version 20 and above, the Name attribute has write access. In versions earlier than 20, the Name attribute does not have write access.

The PLCTag table contains the following actions as given below.

Name	Return type	Description
Delete	void	Deletes the instance. Throws an exception if IsDefault is true.
Export	void	Exports the Simatic ML of a Plc tag table.
ShowInEditor	void	Shows the tag table in the Plc tag table editor.

Program code

Modify the following program code to query the information for a PLC tag table:



```
private static void AccessPlcConstantsUsingFind(PlcTagTable
tagTable)
{
    PlcUserConstantComposition plcUserConstants =
tagTable.UserConstants;
    PlcUserConstant plcUserConstant =
plcUserConstants.Find("Constant XYZ");
    //PlcSystemConstantComposition plcSystemConstants =
tagTable.SystemConstants;
    //PlcSystemConstant plcSystemConstant =
plcSystemConstants.Find("Constant XYZ");
}
private static void EnumeratePlcTags(PlcTagTable tagTable)
{
    PlcTagComposition plcTags = tagTable.Tags;
    foreach (PlcTag plcTag in plcTags)
    {
        string name = plcTag.Name; string typeName =
plcTag.DataTypeName;
        string logicalAddress = plcTag.LogicalAddress;
    }
}
private static void EnumeratePlcTagsUsingFind(PlcTagTable
tagTable)
{
    PlcTagComposition plcTags = tagTable.Tags;
    PlcTag plcTag = plcTags.Find("Constant XYZ");
}
```

2.9 Reading the time of the last changes of a PLC tag table

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
  See Connecting to the TIA Portal

- A project is open.
  See Opening a project

Application

The format of the time stamp is UTC.

Program code

Modify the following program code to read the time stamp of a specific PLC tag table:

```
Copies the following program code to clipboard.

//Reads Time-Stamp of a plc Tag Table
private static void
GetLastModificationDateOfTagtable(PlcSoftware plcSoftware)
{
    PlcTagTable plcTagTable =
plcSoftware.TagTableGroup.TagTables.Find("MyTagTable");
    DateTime modifiedTagTableTimeStamp =
plcTagTable.ModifiedTimeStamp;
}
```

2.10 Deleting a PLC tag table from a group

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
  See Connecting to the TIA Portal
- A project is open.
  See Opening a project

Program code

Modify the following program code to delete a specific tag table from a group:

```
Copies the following program code to clipboard.

//Deletes a PlcTagTable of a group
private static void DeletePlcTagTableInAGroup(PlcSoftware
plcSoftware)
{
```

```
    PlcTagTableSystemGroup group = plcSoftware.TagTableGroup;
    PlcTagTable tagtable =
group.TagTables.Find("MyTagTable");
    if (tagtable!= null)
    {
        tagtable.Delete();
    }
}
```

2.11 Enumerating PLC tags

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
  See Connecting to the TIA Portal
- A project is open.
  See Opening a project

Program code: Enumerating PLC tags in tag tables

Modify the following program code to enumerate all PLC tags in a tag table:

```
 Copies the following program code to clipboard.
//Enumerates all plc tags in a specific tag table
private static void EnumerateAllPlcTagsInTagTable(PlcSoftware
plcSoftware)
{
    PlcTagTable tagTable =
plcSoftware.TagTableGroup.TagTables.Find("Tagtable XYZ");
    foreach (PlcTag tag in tagTable.Tags)
    {
        // add code here
    }
}
```

2.12 Accessing PLC tags

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
  See Connecting to the TIA Portal
- A project is open.
  See Opening a project

Application

The type `PlcTagComposition` represents a collection of plc tags.

Program code: Accessing a specific PLC tag

Modify the following program code to access the required PLC tag. You have access to the following attributes:

Attribute name	Data type	Access
DataTypeName	string	Read and Write
ExternalAccessible	bool	Read and Write
ExternalVisible	bool	Read and Write
ExternalWritable	bool	Read and Write
LogicalAddress	string	Read and Write
Name[1]	string	Read and Write
IsSafety	bool	Read and Write

Note

In TIA Portal Openness version 20 and above, the Name attribute has write access. In versions earlier than 20, the Name attribute does not have write access.



```
Copies the following program code to clipboard.
//Gives individual access to a specific plc tag
private static void AccessPlcTag(PlcTagTable tagTable)
{
    PlcTag tag = tagTable.Tags.Find("Tag XYZ");
```

```
    // The parameter specifies the name of the tag
}
```

Program code: Creating tags

Modify the following program code:

```
Copies the following program code to clipboard.
private static void CreateTagInPLCTagtable(PlcSoftware
plcsoftware)
// Create a tag in a tag table with default attributes
{
    string tagName = "MyTag";
    PlcTagTable table =
plcsoftware.TagTableGroup.TagTables.Find("myTagTable");
    PlcTagComposition tagComposition = table.Tags;
    PlcTag tag = tagComposition.Create(tagName);
}
```

Modify the following program code:

```
Copies the following program code to clipboard.
private static void CreateTagInPLCTagtable(PlcSoftware
plcsoftware)
// Create a tag of data type bool and logical address not set
{
    string tagName = "MyTag";
    string dataType = "Bool";
    string logicalAddress ="";
    PlcTagTable table =
plcsoftware.TagTableGroup.TagTables.Find("myTagTable");
    PlcTagComposition tagComposition = table.Tags;
    PlcTag tag = tagComposition.Create(tagName, dataType,
logicalAddress);
}
```

Program code: Deleting tags

Modify the following program code:

```
Copies the following program code to clipboard.

private static void DeleteTagFromPLCTagtable(PlcSoftware
plcsoftware)
// Deletes a single tag of a tag table
{
    string tagName = "MyTag";
    PlcTagTable table =
plcsoftware.TagTableGroup.TagTables.Find("myTagTable");
    PlcTagComposition tagComposition = table.Tags;
    PlcTag tag = tagComposition.Find(tagName);
    if (tag != null)
    {
        tag.Delete();
    }
}
```

2.13 Accessing PLC constants

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
  See Connecting to the TIA Portal
- A project is open.
  See Opening a project

Application

The type `PlcUserConstantComposition` represents a collection of plc user constants. You have access to the following attributes:

- Name (read only)
- Data type name
- Value

The type `PlcSystemConstantComposition` represents a collection of plc system constants. You have access to the following attributes:

- Name (read only)
- Data type name (read only)
- Value (read only)

Program code: Creating user constants

Modify the following program code:



```
Copies the following program code to clipboard.
private static void CreateUserConstantInPLCTagtable(PlcSoftware plcsoftware)
// Create a user constant in a tag table
{
string constantName = "MyConstant";
PlcTagTable table = plcsoftware.TagTableGroup.TagTables.Find("myTagTable");
PlcUserConstantComposition userConstantComposition = table.UserConstants;
PlcUserConstant userConstant = userConstantComposition.Create(constantName);
}
```

Program code: Deleting user constants

Modify the following program code:



```
Copies the following program code to clipboard.
private static void DeleteUserConstantFromPLCTagtable(PlcSoftware plcsoftware)
// Deletes a single user constant of a tag table
{
PlcTagTable table = plcsoftware.TagTableGroup.TagTables.Find("myTagTable");
PlcUserConstantComposition userConstantComposition = table.UserConstants;
```

```
PlcUserConstant userConstant =
userConstantComposition.Find("MyConstant");
if (userConstant != null)
{
userConstant.Delete();
}
}
```

Program code: Accessing system constants

Modify the following program code:



```
Copies the following program code to clipboard.
//Gives individual access to a specific system constant
private static void AccessSystemConstant(PlcTagTable tagTable)
{
PlcTag systemConstant = tagTable.SystemConstants.Find("Constant
XYZ");
// The parameter specifies the name of the tag
}
```

See also

Creating user-defined groups for PLC tags
Deleting user-defined groups for PLC tags
Deleting a PLC tag table from a group
Accessing PLC tags
Starting the "PLC Tags" editor
Reading the time of the last changes of a PLC tag table